

Tema 2

Personalización avanzada de gráficos

Carlos Castilla Ruiz

c.castilla@upm.es

Índice

1. Estructura de la personalización avanzada
2. Control total del lienzo con la API orientada a objetos
3. Mapeos semánticos limpios y eliminación de bordes
4. Eliminación de leyendas con anotaciones directas

Índice

1. Estructura de la personalización avanzada
2. Control total del lienzo con la API orientada a objetos
3. Mapeos semánticos limpios y eliminación de bordes
4. Eliminación de leyendas con anotaciones directas

1. Estructura de la personalización avanzada

La interfaz orientada a objetos

Gestión explícita de figuras y ejes con subplots.

El uso estricto de **fig**, **ax = plt.subplots()**.

Por qué abandonar la interfaz implícita pyplot (p.ej. **plt.plot()** o **plt.title()** **usa el último gráfico en memoria**).

Gestión coordinada de **múltiples subplots en un lienzo único**.

Control individualizado de límites, ticks y dimensiones.

Mapeos semánticos y limpieza

Ajuste bivariado y **eliminación del ruido visual** del lienzo / canvas.

Control semántico del color con hue, tamaño con size y estilo con style.

Principio de tamaño proporcional y reducción de carga cognitiva.

Eliminando el marco redundante: **sns.despine()**.

Configuración manual de los grids (rejillas) secundarios atenuados.

Anotaciones directas y hallazgos clave (insights)

Liberación de leyendas complejas a través del texto semántico.

Por qué las leyendas tradicionales saturan la memoria icónica.

Mapeo espacial de **texto directo usando ax.text() y ax.annotate()**.

Focalización atencional: teñir de **gris neutro y destacar con color rojo** (o similar) **de énfasis**.

El valor explicativo de las **flechas y llamadas al dato específico**.

Índice

1. Estructura de la personalización avanzada
2. Control total del lienzo con la API orientada a objetos
3. Mapeos semánticos limpios y eliminación de bordes
4. Eliminación de leyendas con anotaciones directas

2. Control total del lienzo con la API orientada a objetos

¿Por qué usar la API orientada a objetos?

La interfaz implícita pyplot es rápida para bocetos breves, pero es un estado global invisible difícil de personalizar y propenso a errores en scripts complejos.

La interfaz OO es explícita: crea **objetos instanciados para la figura (fig, el contenedor global del lienzo) y uno o más ejes (ax, el área de dibujo individual)**.

Estructura recomendada: `fig, ax = plt.subplots(figsize=(W, H))`. Todo **cambio visual** se invoca de forma estructurada **a través de llamadas de método en el objeto ax** (ej. `ax.set_title()`, `ax.set_xlim()`).

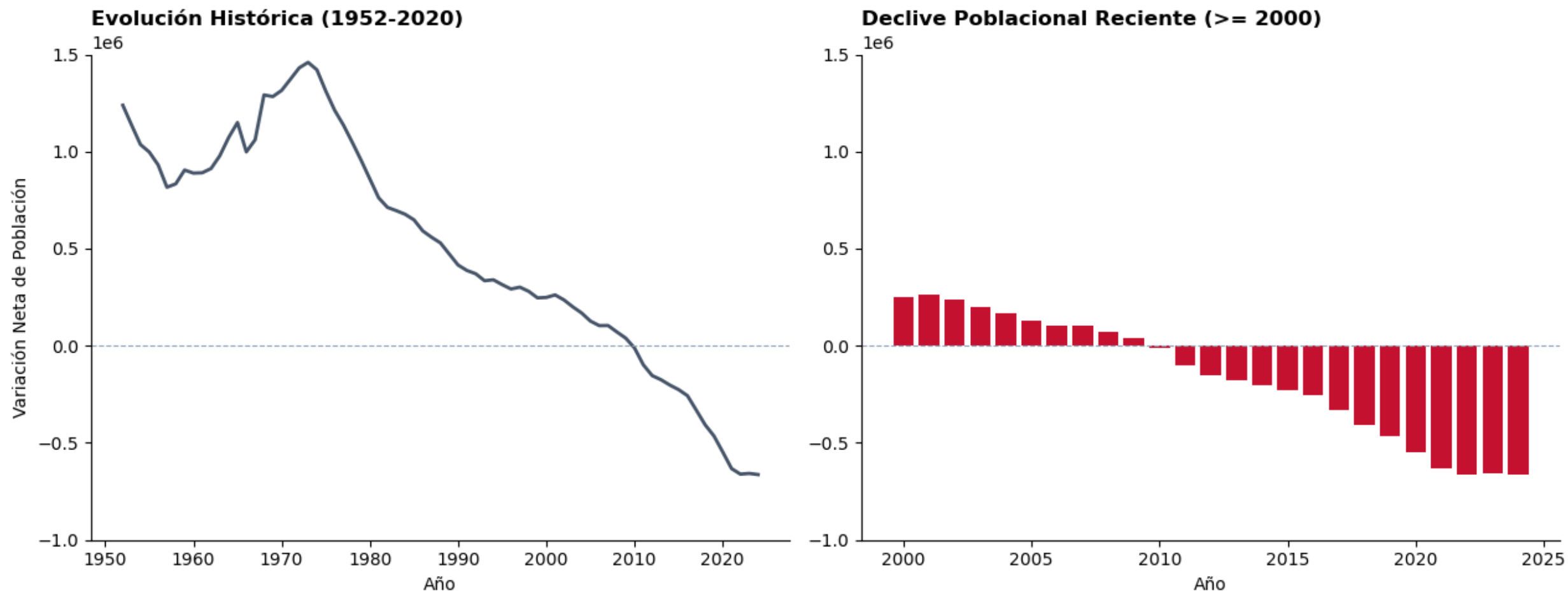
Multigráficos coordinados: `plt.subplots(nrows, ncols)` devuelve una **matriz de ejes ax** sobre los cuales se puede iterar o indexar de forma matemática (ej. `axes[0, 1].plot()`) para pintar paneles paralelos limpios.

Patrón de código estándar de la sesión 3

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

df = pd.read_csv('japan-population.csv')
# 1. Instanciación explícita del lienzo
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# 2. Control de la primera subtrama (ax = axes[0])
axes[0].plot(df['year'], df['pop_var'], color='#475569',
linewidth=2)
axes[0].set_title('Evolución Histórica (1952-2020)',
loc='left', fontsize=12, weight='bold')
...
# 3. Control de la segunda subtrama (ax = axes[1])
df_recent = df[df['year'] >= 2000]
axes[1].bar(df_recent['year'], df_recent['pop_var'],
color='#C51130', edgecolor='none')
...
```



Índice

1. Estructura de la personalización avanzada
2. Control total del lienzo con la API orientada a objetos
3. Mapeos semánticos limpios y eliminación de bordes
4. Eliminación de leyendas con anotaciones directas

3. Mapeos semánticos limpios y eliminación de bordes

Control semántico del color, forma y tamaño

Mapeos en **Seaborn**: el parámetro **hue** aplica **colores categóricos**, **size** modifica el grosor/diámetro y **style** cambia la **forma del marcador**. Permite **mapear hasta 3 dimensiones independientes de forma compacta**.

El peligro del ruido visual: **añadir demasiadas dimensiones semánticas abruma** la memoria de trabajo de la **audiencia**. Si usas **hue**, mantén **size** y **style** apagados a menos que sea estrictamente necesario para accesibilidad.

Teñir con moderación: reserva **paletas categóricas** de **alto contraste** únicamente para **segmentaciones relevantes** del negocio y opta por **paletas secuenciales** para **variaciones cuantitativas continuas**.

Ajuste estricto del **despine**: por defecto, Matplotlib encierra el gráfico en una caja con 4 bordes. El **borde superior y derecho no aportan valor informativo**. `sns.despine(ax=ax)` los elimina, aligerando el gráfico.

Ejemplo: mapeo y despine avanzado

```
fig, ax = plt.subplots(figsize=(10, 6))
```

```
# 1. Mapeo semántico controlado con Seaborn
sns.scatterplot(data=df, x='Logged GDP per capita', y='Ladder score', hue='Regional indicator', size='Social support', palette='Set2', sizes=(30, 250), alpha=0.85, ax=ax)
```

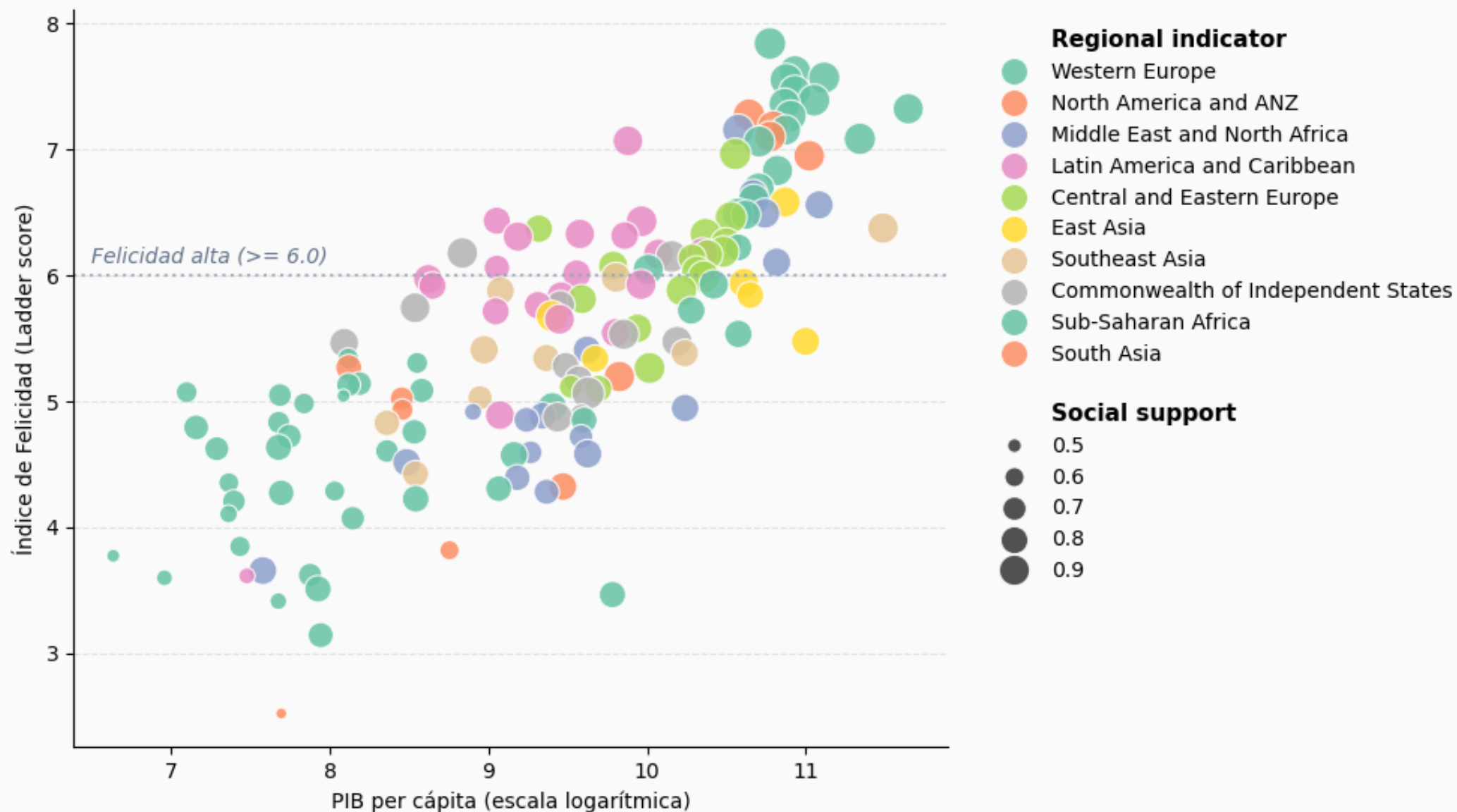
```
# 2. Eliminación de bordes e integración de rejilla / grid sutil
```

```
sns.despine(ax=ax, top=True, right=True)
ax.grid(True, axis='y', linestyle='--', alpha=0.3)
```

```
# 3. Control y colocación de leyenda fuera del gráfico
```

```
ax.legend(bbox_to_anchor=(1.03, 1), loc='upper left', frameon=False, alignment='left')
```

Relación Bivariada: Riqueza (PIB) frente a Felicidad Nacional



Índice

1. Estructura de la personalización avanzada
2. Control total del lienzo con la API orientada a objetos
3. Mapeos semánticos limpios y eliminación de bordes
4. Eliminación de leyendas con anotaciones directas

4. Eliminación de leyendas con anotaciones directas

El problema de las leyendas tradicionales

Sobrecarga visual: forzar al usuario a **mirar** el gráfico, luego saltar **a una leyenda de caja externa** para identificar los colores, y luego volver a saltar al gráfico cansa y **reduce la comprensión del dato**.

Solución: el uso de **anotaciones de texto directas** al final de cada serie de datos o encima de las barras elimina por completo la necesidad de una leyenda externa compleja.

En **Python**: **ax.text**(x, y, 'Texto', ...) o **ax.annotate**('Texto', xy=(x, y), xytext=(x_txt, y_txt), arrowprops=...) para colocar etiquetas con precisión espacial absoluta.

Focalización: pintar las **series de datos** en un **gris neutro atenuado** y destacar la **serie de interés** en un **color fuerte**, anotando su valor de forma directa.

Ejemplo: Anotación y etiquetado directo

```
fig, ax = plt.subplots(figsize=(9, 5))
```

```
sns.barplot(data=df_top, x='salary', y='state', ax=ax, color='#94A3B8')
```

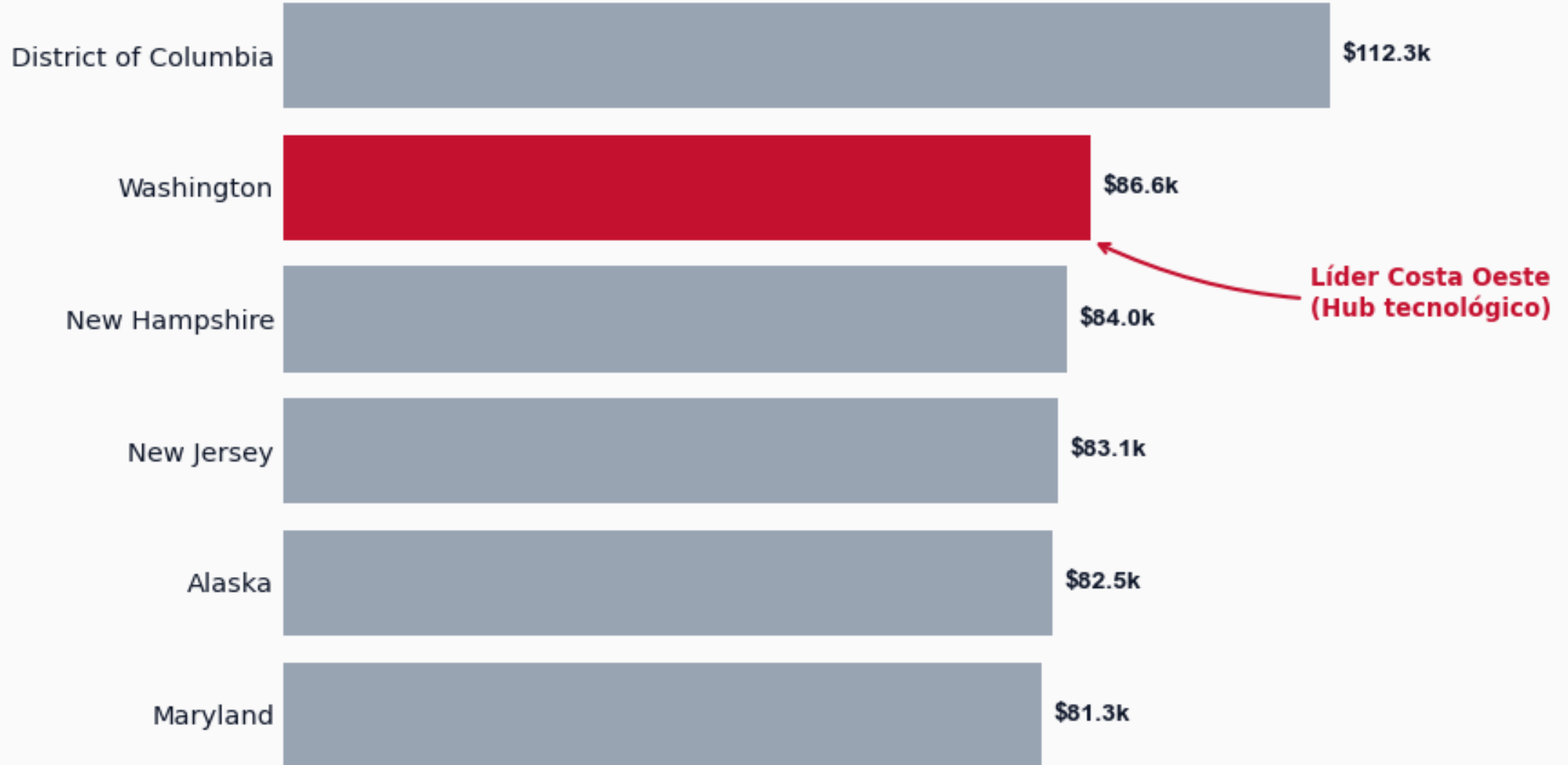
```
# 1. Teñir de énfasis el elemento clave de forma manual  
ax.patches[1].set_facecolor('#C51130') # Washington
```

```
# 2. Añadir anotaciones al final de las barras  
for bar in ax.patches:
```

```
    width = bar.get_width()  
    ax.text(width + 1.2, bar.get_y() + bar.get_height()/2,  
            f'${width:.1f}k', va='center', ha='left',  
            fontname='Arial', fontsize=10, fontweight='bold')
```

```
# 3. Anotación directa descriptiva con flecha  
ax.annotate('Líder', xy=(x_t, y_t), xytext=(110, 1.6),  
            arrowprops=dict(facecolor='#C51130', arrowstyle='->'))
```

Distribución Salarial en EE.UU.: Washington destaca tras la capital federal



Tema 2

Personalización avanzada de gráficos

Carlos Castilla Ruiz

c.castilla@upm.es
